

Deep Network Development

Autumn 2026, Practice #5

Viktor Varga
Department of Artificial Intelligence, ELTE IK

Last week - Multidimensional array example

The ndarray type – Main attributes:

```
self.board = np.zeros((9, 9), dtype=np.uint8)
```

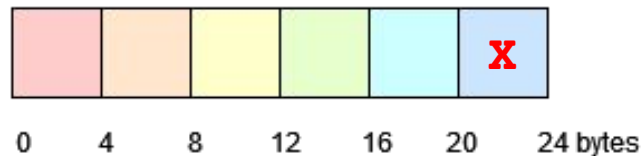
- ndarray.size 81
- ndarray.ndim 2
- ndarray.shape (9, 9)
- ndarray.dtype uint8
- ndarray.itemsize 1
- ndarray.strides
- ndarray.base

Last week - Multidimensional arrays

```
a = np.zeros((2, 3), dtype=np.int64)    # a.shape == (2, 3)
print(type(a))                          # <class 'numpy.ndarray'>
print(a)                                # [[0 0 0]
                                       #  [0 0 0]]
```



as seen



in memory

E.g., the memory address of `a[1, 2]`, will be the address of `a[0, 0]` plus $(1*3 + 2*1)*a.itemsize$ bytes.

Last week - Array indexing

I

Indexing an individual element of the nD array.
Returns a scalar.

a [1, 2]

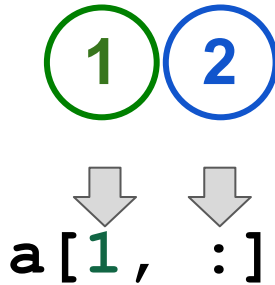
Indexing of an individual element from an array:

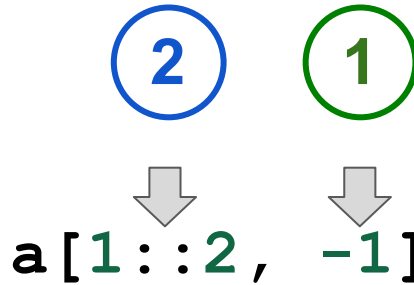
- **All axes** of an array are indexed using an **individual index**.
- A single element is returned as a **scalar**.

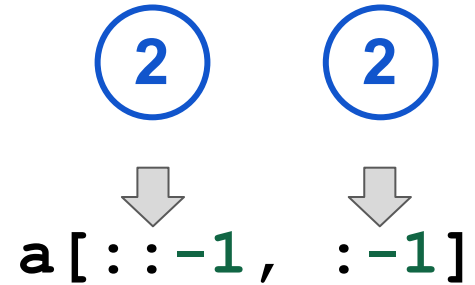
Last week - Array indexing

(B)

Basic indexing.
Returns a view array over the original array.*


`a[1, :]`


`a[1::2, -1]`


`a[:, :-1, :-1]`

Basic indexing:

- Can only use a **combination of individual indexing and slicing**.
- **At least one slicing** is needed.**
- An array is returned which is a view over the original array.*
- Each **axis with individual index** will be **collapsed** in the result.
- Each **axis with slicing**** will be **kept** in the result (even if length is zero).

* Always true for base arrays. Often true for view arrays, but in certain cases not.

** Can be implicit.

Last week - Array indexing



Basic indexing.
Returns a view array over
the original array.*

a[2, :]

[0]	[1]	[2]	[3]	[4]

a[:, :2]

[0,0]	[0,1]			
[1,0]	[1,1]			
[2,0]	[2,1]			
[3,0]	[3,1]			

a[:, 2:]

		[0,2]	[0,1]	[0,2]
		[1,2]	[1,1]	[1,2]
		[2,2]	[2,1]	[2,2]
		[3,2]	[3,1]	[3,2]

a[2:3, :]

[0,0]	[1,0]	[2,0]	[3,0]	[4,0]

a[:2, 2:-1]

		[0,0]	[0,1]	
		[1,0]	[1,1]	

a[:3:2, -1::-2]

[0,2]		[0,1]		[0,0]
[1,2]		[1,1]		[1,0]

Last week - Array indexing



Basic indexing.
Returns a view array over
the original array.*

Implicit slicing - Syntactic sugar

`a[0]` is equivalent to `a[0]` or `a[0, :]` or `a[0, :, :]` or ...

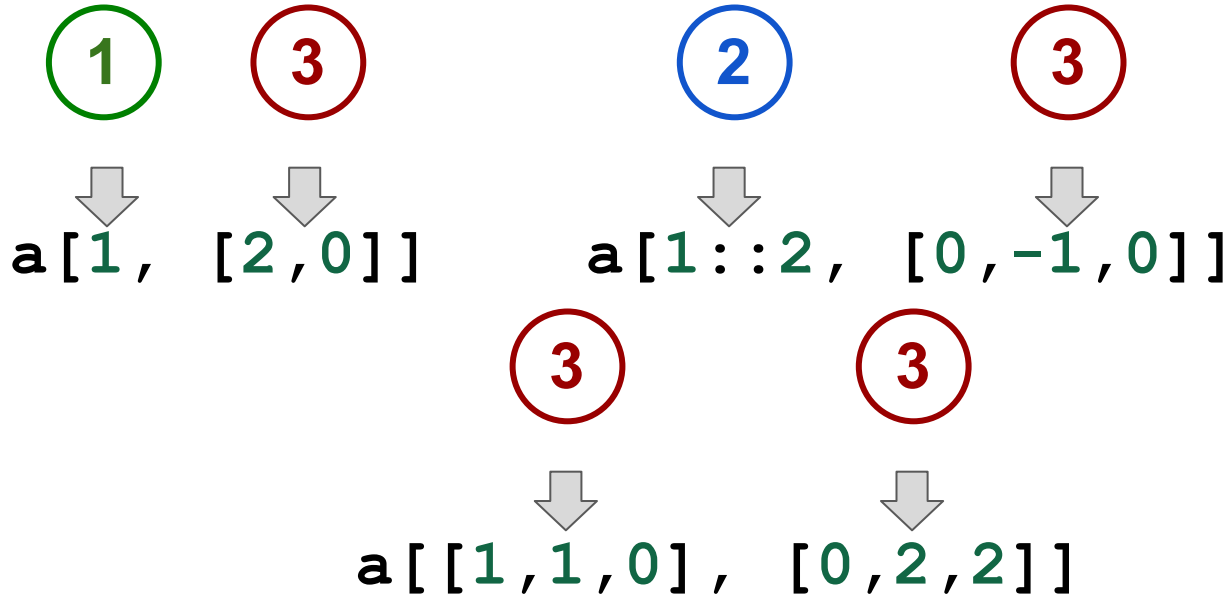
`a[... , -2]` is equivalent to `a[-2]` or `a[:, -2]` or `a[:, :, -2]` or ...

... depending on `a.ndim`

Last week - Array indexing

A

Advanced indexing.
Returns a new array.



Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

Last week - Array indexing



Advanced indexing.
Returns a new array.

a[[1,3,0], [2,-2,0]]

[2]				
		[0]		
			[1]	

a[2, [1,-1]]

	[0]			[1]

a[[1,3,0], -1::-2]

[2,2]		[2,1]		[2,0]
[0,2]		[0,1]		[0,0]
[1,2]		[1,1]		[1,0]

Last week - Array indexing

①
↓
`a[1, [False, True, True]]`

④
↓

④
↓
`a[[True, True], :]`

②
↓

④
↓

`a[[[False, True, True], [False, True, True]]]`

Ⓐ

Advanced indexing.
Returns a new array.

Advanced indexing:

- Will happen if either integer sequence or mask indexing is used at least once.
- A **new array** is returned with the selected elements copied into it.

Last week - Array indexing



Advanced indexing.
Returns a new array.

m.shape: (2, 3)

False	True	True
False	True	False

a.shape: (2, 3)

0	1	2
3	4	5

a[m].shape: (3,)



1	2	4
---	---	---

mc.shape: (2,)

False
True

0	1	2
3	4	5

a[mc, :].shape: (1, 3)



3	4	5
---	---	---

mr.shape: (3,)

True	False	True
------	-------	------

0	1	2
3	4	5

a[:, mr].shape: (2, 2)



0	2
3	5

Last week - Array indexing

I

Indexing an individual element of the nD array.
Returns a scalar.

B

Basic indexing.
Returns a view array over the original array.*

A

Advanced indexing.
Returns a new array.

Easy to remember: If an indexing of an array is **not** individual element indexing **or** basic indexing, **then it is advanced indexing.**

Last week - Array indexing for **writing**

```
a = np.array([[0, 1, 2], [10, 11, 12]], dtype=np.int64) # a.shape == (2, 3)
```

```
my_array[:, [0, 2]] = [[42, 99], [77, 81]]
```

Matching shapes on the two sides: (2, 2) v (2, 2)



```
my_array[:, -1, 1:] = 99
```

Compatible shapes on the two sides: (2, 2) v (1,) / scalar



**BROADCASTING
OK**

```
my_array[[1, 0], :2] = 42, 99, 77, 81
```

Not compatible shapes on the two sides: (2, 2) v (4,)



**BROADCASTING
FAILS**

Last week - Element-wise ops & broadcasting, (m, n) vs. (1,)

... or the shapes need to be compatible in terms of broadcasting.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
```

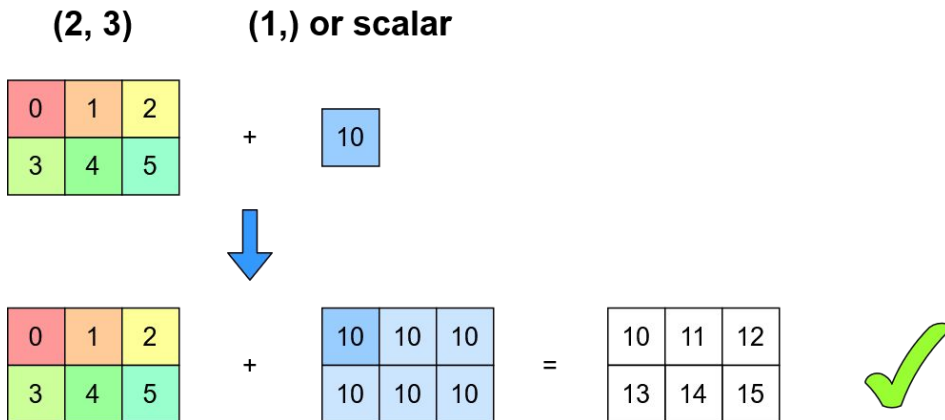
```
print((a + 10).shape)
```

```
# --> (2, 3)
```

```
print(a + 10)
```

```
# --> [[10 11 12]
```

```
#      [13 14 15]]
```

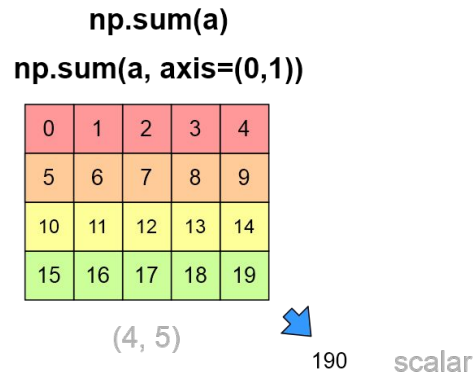
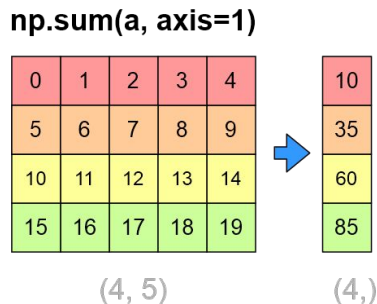
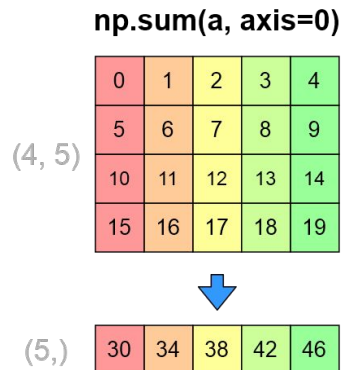


Last week - Collapse operations (with an axis parameter)

Collapse operations can collapse only selected axes of an array.

```
a = np.arange(20, dtype=np.int64).reshape(4, 5)    # a.shape == (4, 5)

print(np.sum(a, axis=0))                            # --> [30 34 38 42 46], a.shape == (5,)
print(np.sum(a, axis=1))                            # --> [10 35 60 85], a.shape == (4,)
print(np.sum(a, axis=(0,1)))                        # --> 190
print(np.sum(a))                                    # --> 190
```



Last week - Reshaping arrays

```
a6 = np.arange(6, dtype=np.int64)           # a6.shape == (6,)  
  
a2x3 = a6.reshape((2, 3))                  # a2x3.shape == (2, 3)  
a3x2 = a2x3.reshape((3, 2))                # a3x2.shape == (3, 2)  
  
print(a6)                                   # [0 1 2 3 4 5]
```

0	1	2	3	4	5
---	---	---	---	---	---

```
print(a2x3)                                  # [[0 1 2]  
                                             # [3 4 5]]
```

0	1	2
3	4	5

```
print(a3x2)                                  # [[0 1]  
                                             # [2 3]  
                                             # [4 5]]
```

0	1
2	3
4	5

NumPy - Reshaping arrays, “-1” axis length

```
a6 = np.arange(6, dtype=np.int64)           # a6.shape == (6,)
```

```
a2x3 = a6.reshape((2, 3))                  # a2x3.shape == (2, 3)
```

0	1	2
3	4	5

```
a6_ = a2x3.reshape((-1,))                  # a6_.shape == (6,)
```



0	1	2	3	4	5
---	---	---	---	---	---

```
a3x2 = a2x3.reshape((3, -1))              # a3x2.shape == (3, 2)
```



0	1
2	3
4	5

Syntactic sugar: NumPy can infer one of the axis lengths automatically.
(Specify a length of -1 for this.)

Exercise

This week's example:

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

Exercise

$n_shops = 4$
 $n_product_types = 3$

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

$(n_shops, n_product_types)$
num_sold_products

"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78

$(n_shops,)$
profit_margin_of_shops

"Swimming suit"	"Raincoat"	"Sweater"
800.	2 250.	1 900.

$(n_product_types,)$
product_prices

"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
91 250.	275 000.	78 000.	45 700.

$(n_shops,)$
cost_of_shops

Exercise

$n_shops = 4$
 $n_product_types = 3$

shop_names_list

"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

product_names_list

"Swimming suit"	"Raincoat"	"Sweater"
-----------------	------------	-----------

($n_shops, n_product_types$)
num_sold_products

800.	2 250.	1 900.
------	--------	--------

($n_product_types$,)
product_prices

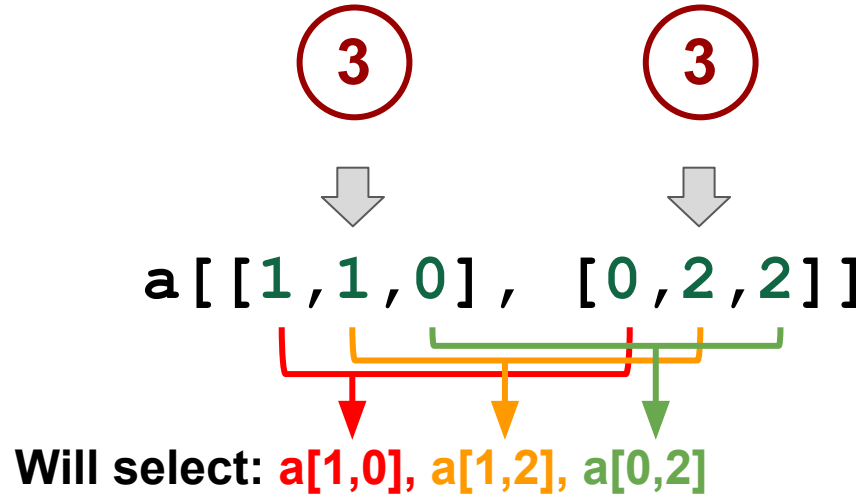
0.45	0.17	0.52	0.78
------	------	------	------

(n_shops ,)
profit_margin_of_shops

91 250.	275 000.	78 000.	45 700.
---------	----------	---------	---------

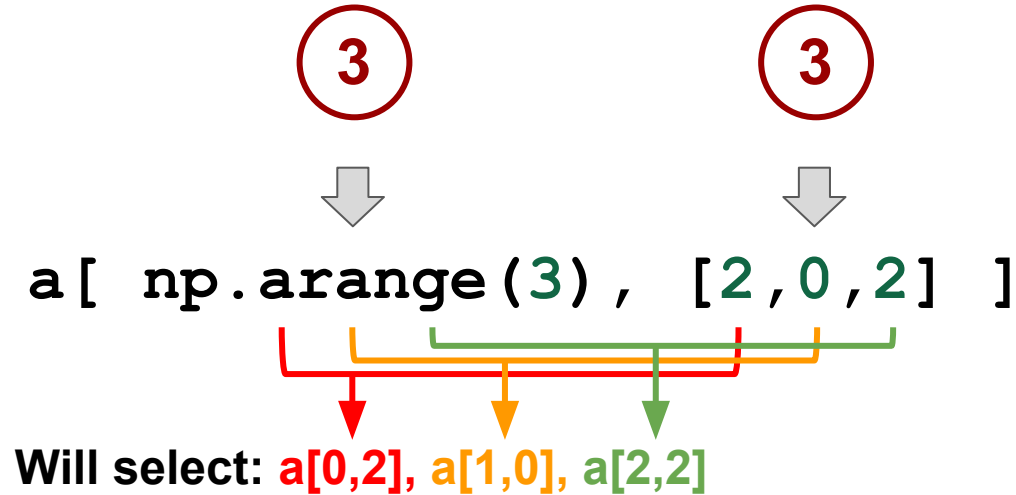
(n_shops ,)
cost_of_shops

NumPy - np.arange() for array indexing



Advanced indexing.
Returns a new array.

NumPy - np.arange() for array indexing



Advanced indexing.
Returns a new array.

NumPy - np.arange() for array indexing

```
best_selling_prod_idx = np.argmax(num_sold_products, axis=1) # (4,)
print( best_selling_prod_idx ) # [1 2 0 2]

best_selling_prod_numbers = num_sold_products[ np.arange(n_shops), best_selling_prod_idx ]
print( best_selling_prod_numbers.shape ) # (4,)
print( best_selling_prod_numbers ) # [ 711  207  196   96]
```

373	711	49
75	182	207
196	17	24
7	3	96

Use case: Select / write exactly one element from each row / column / slice

NumPy - np.arange() for array indexing

```
best_selling_prod_idx = np.argmax(num_sold_products, axis=1)      # (4,)
print( best_selling_prod_idx )                                     # [1 2 0 2]

best_selling_prod_numbers = num_sold_products[ np.arange(n_shops), best_selling_prod_idx ]
print( best_selling_prod_numbers.shape )                          # (4,)
print( best_selling_prod_numbers )                               # [ 711  207  196   96]
```

Will select: $a[0,1]$, $a[1,2]$, $a[2,0]$, $a[3,2]$

373	711	49
75	182	207
196	17	24
7	3	96

NumPy - Concatenating arrays

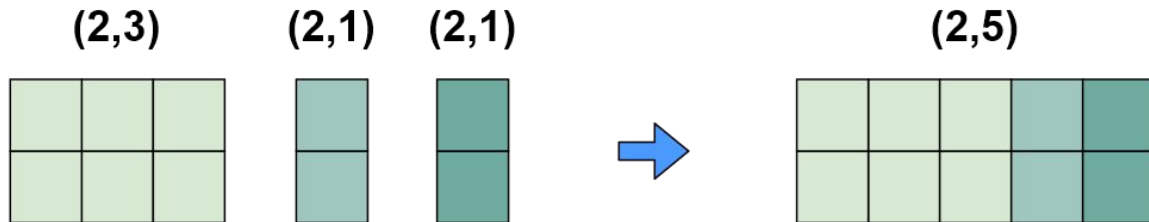
```
arr1 = np.zeros((2,3), dtype=np.int32)
arr2 = np.ones((2,1), dtype=np.float64)
arr3 = np.full((2,1), dtype=np.uint8, fill_value=42)
```

```
cat_arr = np.concatenate([arr1, arr2, arr3], axis=1)
```

```
print( cat_arr )           # [[ 0.  0.  0.  1. 42.]
                           # [ 0.  0.  0.  1. 42.]]
print( cat_arr.shape )     # (2, 5)
print( cat_arr.dtype )     # float64
```

np.concatenate:
Joins multiple arrays
along an **existing axis**.

The contents of the
argument arrays are
copied into a new array.



NumPy - Concatenating arrays

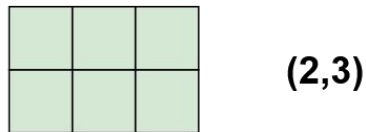
```
arr1 = np.zeros((2,3), dtype=np.int32)
arr2 = np.ones((2,1), dtype=np.float64)
arr3 = np.full((2,1), dtype=np.uint8, fill_value=42)
```

```
cat_arr = np.concatenate([arr1, arr2, arr3], axis=1)
```

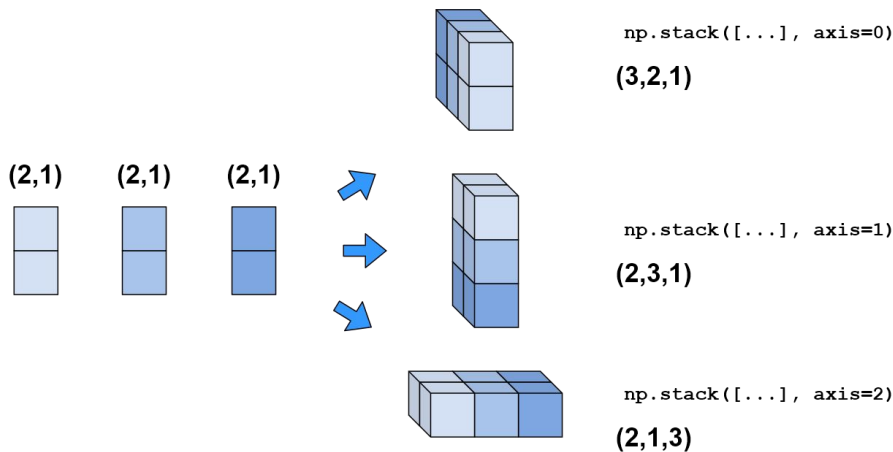
```
print( cat_arr )           # [[ 0.  0.  0.  1. 42.]
                           #  [ 0.  0.  0.  1. 42.]]
print( cat_arr.shape )     # (2, 5)
print( cat_arr.dtype )     # float64
```

```
cat_arr = np.concatenate([arr1, arr2, arr3], axis=0)
# -> ValueError: all the input array dimensions except
for the concatenation axis must match exactly, but ...
```

`np.concatenate([...], axis=0)`



NumPy - Concatenating arrays



`np.stack`: Joins multiple arrays along a new axis.

The contents of the argument arrays are copied into a new array.

```
arr1 = np.zeros((2,1), dtype=np.int32)
arr2 = np.ones((2,1), dtype=np.float64)
arr3 = np.full((2,1), dtype=np.uint8, fill_value=99)
```

```
stacked1 = np.stack([arr1, arr2, arr3], axis=0)
print(stacked1)
# [[[ 0.]
#      [ 0.]]
#     [[ 1.]
#      [ 1.]]
#     [[99.]
#      [99.]]]
```

```
print(stacked1.shape) # (3, 2, 1)
```

```
stacked2 = np.stack([arr1, arr2, arr3], axis=1)
print(stacked2)
# [[[ 0.]
#      [ 1.]
#      [99.]]
#     [[ 0.]
#      [ 1.]
#      [99.]]]
```

```
print(stacked2.shape) # (2, 3, 1)
```

```
stacked3 = np.stack([arr1, arr2, arr3], axis=2)
print(stacked3)
# [[[ 0.  1. 99.]
#      [ 0.  1. 99.]]]
print(stacked3.shape) # (2, 1, 3)
```

NumPy - Concatenating arrays

```
arr1 = np.zeros((2,3), dtype=np.int32)
arr2 = np.ones((2,1), dtype=np.float64)
arr3 = np.full((2,1), dtype=np.uint8, fill_value=42)
```

```
cat_arr = np.stack([arr1, arr2, arr3], axis=1)
```

```
print(cat_arr)
```

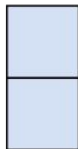
```
# -> ValueError: all input arrays must have the same shape
```

np.stack: Joins multiple arrays along a **new axis**.

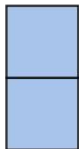
The contents of the argument arrays are copied into a new array.

`np.stack([...], axis=...)`

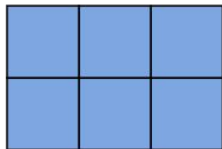
(2,1)



(2,1)

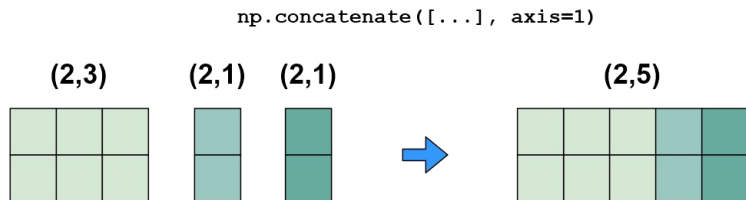


(2,3)

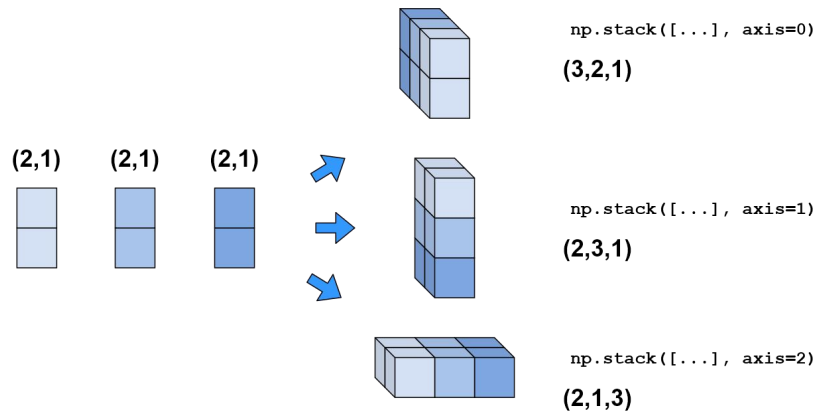


NumPy - Concatenating arrays

np.concatenate:
Joins multiple arrays
along an **existing axis**.



np.stack: Joins
multiple arrays along a
new axis.



NumPy - Concatenating arrays

np.concatenate:
Joins multiple arrays
along an **existing axis**.

np.stack: Joins
multiple arrays along a
new axis.

Both of these operations create **new arrays**, similarly to **ndarray.append()**.

`np.concatenate(..., axis=1)`

(2,3) (2,1) (2,1) **Do not concatenate / stack in a loop ($O(n^2)$)!**

Instead, either:

- Collect the smaller arrays in a list and concatenate them at the end.
- If the total size is known, create the large array in advance and write into it.

`np.stack(..., axis=1)`

(3,2,1)

`np.stack(..., axis=2)`

(2,3,1)

(4,1,3)

Exercise - Total profit

n_shops = 4
n_product_types = 3

```
revenues = num_sold_products * product_prices
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

(n_shops, n_product_types)

???

×

"Swimming suit"	"Raincoat"	"Sweater"
800.	2 250.	1 900.

(n_product_types,)

NumPy - Element-wise ops & broadcasting, (m, n) vs. (1,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
```

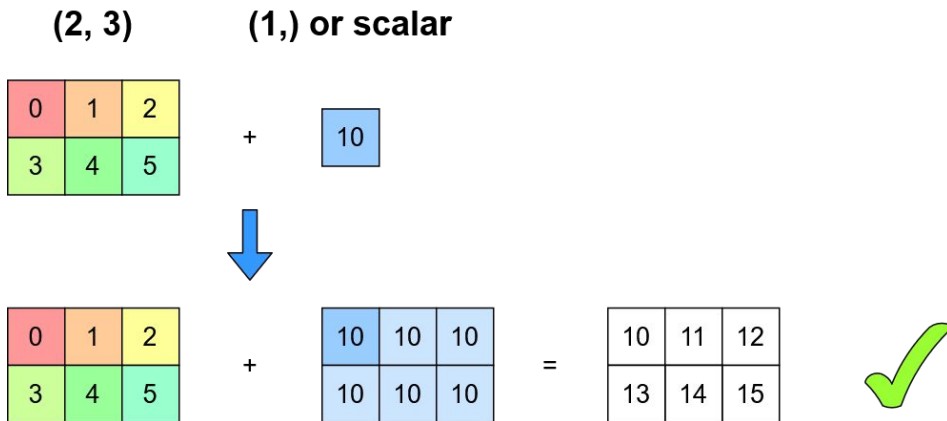
```
print((a + 10).shape)
```

```
# --> (2, 3)
```

```
print(a + 10)
```

```
# --> [[10 11 12]
```

```
#      [13 14 15]]
```

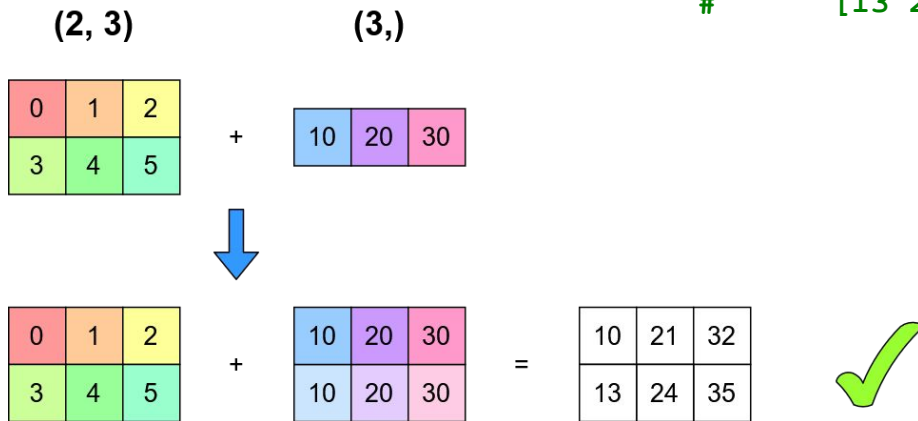


NumPy - Element-wise ops & broadcasting, (m, n) vs. (n,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20, 30], dtype=np.int64)              # b.shape == (3,)
```

```
print((a + b).shape)                                     # --> (2, 3)
print(a + b)                                              # --> [[10 21 32]
                                                         #       [13 24 35]]
```



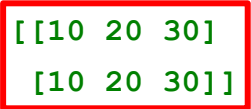
NumPy - Element-wise ops & broadcasting, (m, n) vs. (n,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

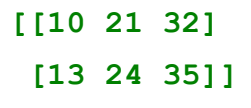
```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20, 30], dtype=np.int64)             # b.shape == (3,)
b2d = np.tile(b, (2, 1))
print(b2d.shape)
print(b2d)

print((a + b2d).shape)
print(a + b2d)
```

--> (2, 3)

--> 

--> (2, 3)

--> 

No need to physically create the repetitions!

Broadcasting is virtual and therefore more efficient than manual tiling / repetitions!

Exercise - Total profit

n_shops = 4
n_product_types = 3

```
revenues = num_sold_products * product_prices
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

(n_shops, n_product_types)

???

×

"Swimming suit"	"Raincoat"	"Sweater"
800.	2 250.	1 900.

(n_product_types,)

Exercise - Total profit

$n_shops = 4$
 $n_product_types = 3$

```
revenues = num_sold_products * product_prices
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	373	711	49
"Reykjavík"	75	182	207
"Tunis"	196	17	24
"Ulaanbaatar"	7	3	96

×

	"Swimming suit"	"Raincoat"	"Sweater"
	800.	2 250.	1 900.

($n_shops, n_product_types$) v ($n_product_types,$)

Exercise - Total profit

```
revenues = num_sold_products * product_prices
```

	Swimming suit	Raincoat	Sweater
Hanoi	373	711	49
Reykjavik	75	182	207
Tunis	196	17	24
Ulaanbaatar	7	3	96

×

Swimming suit	Raincoat	Sweater
800.	2 250.	1 900.
800.	2 250.	1 900.
800.	2 250.	1 900.
800.	2 250.	1 900.

→

Swimming suit	Raincoat	Sweater
298 400.	1 599 750.	93 100.
60 000.	409 500.	393 300.
156 800.	38 250.	45 600.
5 600.	6 750.	182 400.

$(n_shops, n_product_types) \times (n_product_types, n_product_types) \rightarrow (n_shops, n_product_types)$

Exercise - Total profit

```
revenues = num_sold_products * product_prices
```

```
gross_profits = revenues * profit_margin_of_shops
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	298 400.	1 599 750.	93 100.
"Reykjavík"	60 000.	409 500.	393 300.
"Tunis"	156 800.	38 250.	45 600.
"Ulaanbaatar"	5 600.	6 750.	182 400.

???

×

"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78

(n_shops, n_product_types) v (n_shops,) → **X**

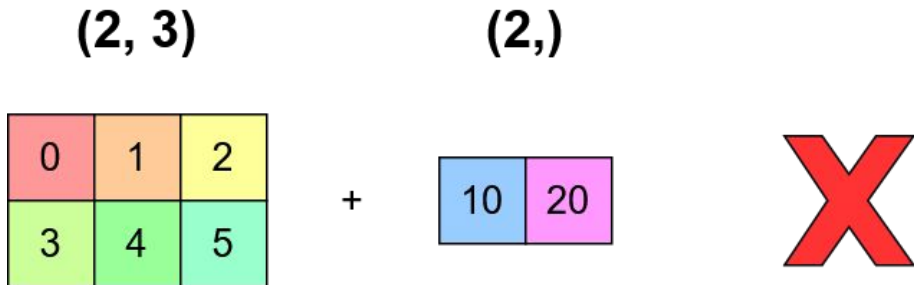
NumPy - Element-wise ops & broadcasting, (m, n) vs. (m,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20], dtype=np.int64)                 # b.shape == (2,)

print(a + b)

# -> ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```



NumPy - Rules of broadcasting

Example:
(2, 3) v (3,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

NumPy - Rules of broadcasting

Example:
 $(2, 3) \vee (3,)$

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

$(2, 3) \vee (1, 3)$

NumPy - Rules of broadcasting

Example:
(2, 3) v (3,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(2, 3) v (1, 3)

2. Check the **lengths of axes** at matching positions!

- a. If they are equal → **OK (no action)**

(2, 3) v (1, 3)

- b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis
to match the axis lengths.**

- c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**

NumPy - Rules of broadcasting

Example:
(2, 3) v (3,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(2, 3) v (1, 3)

2. Check the **lengths of axes** at matching positions!

- a. If they are equal → **OK (no action)**

(2, 3) v (1, 3)

- b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis
to match the axis lengths.**

→ (2, 3) v (2, 3)

- c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**

NumPy - Rules of broadcasting

Example:
(2, 3) v (3,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(2, 3) v (1, 3)

2. Check the **lengths of axes** at matching positions!

- a. If they are equal → **OK (no action)**

(2, 3) v (1, 3)

- b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis
to match the axis lengths.**

→ (2, 3) v (2, 3)

- c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**



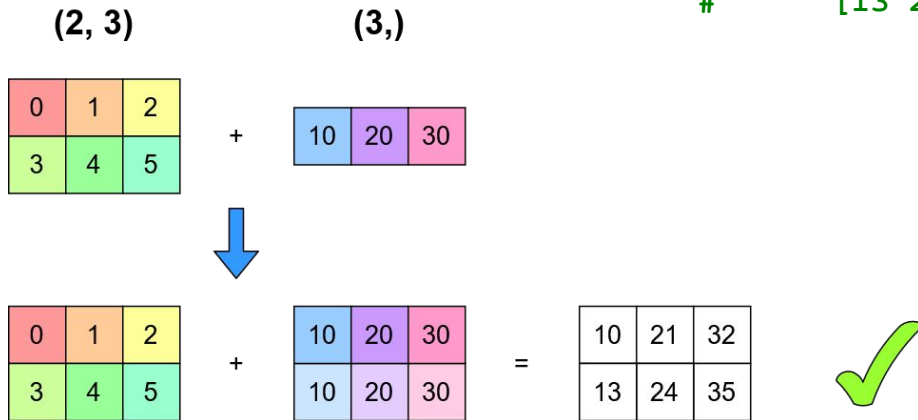
Broadcasting succeeds!
Common shape: (2, 3)

NumPy - Element-wise ops & broadcasting, (m, n) vs. (n,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20, 30], dtype=np.int64)              # b.shape == (3,)
```

```
print((a + b).shape)                                     # --> (2, 3)
print(a + b)                                             # --> [[10 21 32]
                                                         #       [13 24 35]]
```



NumPy - Rules of broadcasting

Example:
(2, 3) v (2,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.
2. Check the **lengths of axes** at matching positions!
 - a. If they are equal → **OK (no action)**
 - b. If they are not equal, but one of them is 1
→ **Perform virtual repetition along this axis
to match the axis lengths.**
 - c. If they are not equal, and none of them is 1
→ **Broadcasting fails.**

NumPy - Rules of broadcasting

Example:
(2, 3) v (2,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(2, 3) v (1, 2)

2. Check the **lengths of axes** at matching positions!
 - a. If they are equal → **OK (no action)**
 - b. If they are not equal, but one of them is 1
→ **Perform virtual repetition along this axis
to match the axis lengths.**
 - c. If they are not equal, and none of them is 1
→ **Broadcasting fails.**

NumPy - Rules of broadcasting

Example:
(2, 3) v (2,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(2, 3) v (1, 2)

2. Check the **lengths of axes** at matching positions!
 - a. If they are equal → **OK (no action)**
 - b. If they are not equal, but one of them is 1
→ **Perform virtual repetition along this axis
to match the axis lengths.**
 - c. If they are not equal, and none of them is 1
→ **Broadcasting fails.**

(2, 3) v (1, 2)

Broadcasting fails!

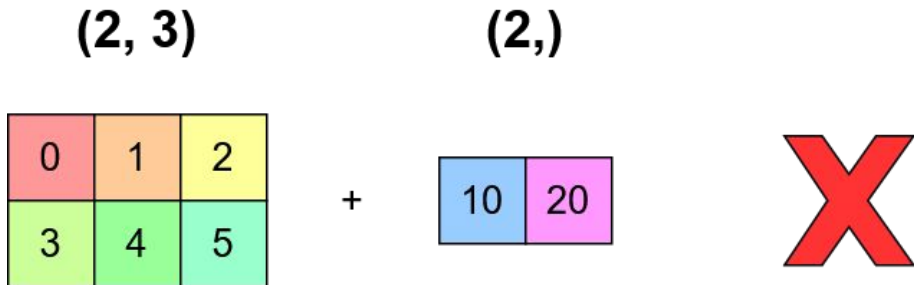
NumPy - Element-wise ops & broadcasting, (m, n) vs. (m,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20], dtype=np.int64)                 # b.shape == (2,)

print(a + b)

# -> ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```



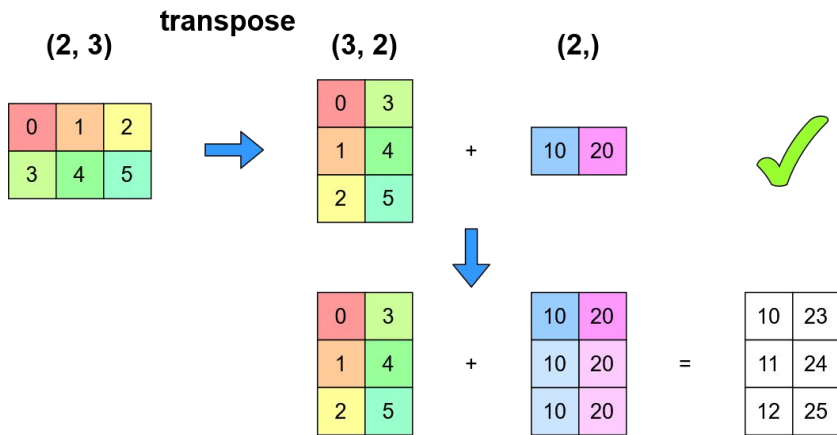
NumPy - Element-wise ops & broadcasting, (n, m) vs. (m,)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)           # a.shape == (2, 3)
b = np.array([10, 20], dtype=np.int64)                         # b.shape == (2,)
```

```
c = a.T + b
print(c.shape)
print(c)
```

```
# c.shape == (3, 2)
# [[10 23]
#  [11 24]
#  [12 25]]
```



NumPy - Swapping the axis order of an array

```
a = np.arange(6).reshape(2, 3)
```

```
print( a.shape )
```

```
print( a )
```

```
# (2, 3)
```

```
# [[0 1 2]
```

```
#  [3 4 5]]
```

0	1	2
3	4	5

```
print( np.swapaxes(a, 0, 1).shape )
```

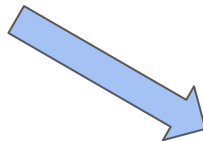
```
print( np.swapaxes(a, 0, 1) )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```



```
print( np.transpose(a).shape )
```

```
print( np.transpose(a) )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```



```
print( a.T.shape )
```

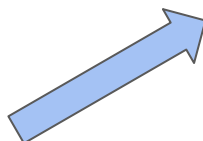
```
print( a.T )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```



0	3
1	4
2	5

The resulting array is
a view of the original one.*

* Always true for base arrays, but not necessarily for views.

NumPy - Swapping the axis order of an array

```
a = np.arange(6).reshape(2, 3)
```

```
print( a.shape )
```

```
print( a )
```

```
# (2, 3)
```

```
# [[0 1 2]
```

```
#  [3 4 5]]
```

0	1	2
3	4	5

```
print( np.swapaxes(a, 0, 1).shape )
```

```
print( np.swapaxes(a, 0, 1) )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```

In 2D, these are equivalent to
matrix transpose.

```
print( np.transpose(a).shape )
```

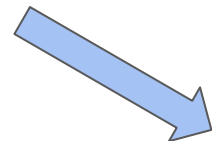
```
print( np.transpose(a) )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```



0	3
1	4
2	5

```
print( a.T.shape )
```

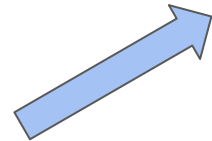
```
print( a.T )
```

```
# (3, 2)
```

```
# [[0 3]
```

```
#  [1 4]
```

```
#  [2 5]]
```



The resulting array is
a view of the original one.*

* Always true for base arrays, but not necessarily for views.

NumPy - Swapping the axis order of an array

```
a = np.arange(24).reshape((2,3,4))  
print( a.shape )                # (2, 3, 4)
```

```
a1 = np.swapaxes(a, 0, 1)  
print( a1.shape )               # (3, 2, 4)
```

```
a2 = np.swapaxes(a, 0, 2)  
print( a2.shape )               # (4, 3, 2)
```

```
a3 = np.swapaxes(a, 1, 2)  
print( a3.shape )               # (2, 4, 3)
```

```
a4 = np.transpose(a, (1, 2, 0))  
print( a4.shape )               # (3, 4, 2)
```

A few examples for higher dimensionality...

NumPy - Rules of broadcasting

Example:
(3, 2) v (2,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(3, 2) v (1, 2)

2. Check the **lengths of axes** at matching positions!

- a. If they are equal → **OK (no action)**

(3, 2) v (1, 2)

- b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis
to match the axis lengths.**

→ (3, 2) v (3, 2)

- c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**



Broadcasting succeeds!
Common shape: (3, 2)

Exercise - Total profit

```
revenues = num_sold_products * product_prices
```

```
revenues_T = revenues.T
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	298 400.	1 599 750.	93 100.
"Reykjavík"	60 000.	409 500.	393 300.
"Tunis"	156 800.	38 250.	45 600.
"Ulaanbaatar"	5 600.	6 750.	182 400.



	"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	298 400.	60 000.	156 800.	5 600.
"Raincoat"	1 599 750.	409 500.	38 250.	6 750.
"Sweater"	93 100.	393 300.	45 600.	182 400.

(n_shops, n_product_types) → (n_product_types, n_shops)

Exercise - Total profit

```
revenues = num_sold_products * product_prices  
revenues_T = revenues.T
```

```
gross_profits = revenues_T * profit_margin_of_shops
```

	"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	298 400.	60 000.	156 800.	5 600.
"Raincoat"	1 599 750.	409 500.	38 250.	6 750.
"Sweater"	93 100.	393 300.	45 600.	182 400.

(n_product_types, n_shops)

???

×

"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78

(n_shops,)

Exercise - Total profit

```
revenues = num_sold_products * product_prices  
revenues_T = revenues.T
```

```
gross_profits = revenues_T * profit_margin_of_shops
```

	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	298 400.	60 000.	156 800.	5 600.
"Raincoat"	1 599 750.	409 500.	38 250.	6 750.
"Sweater"	93 100.	393 300.	45 600.	182 400.

×

"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78

(n_product_types, n_shops) v (n_shops,)

Exercise - Total profit

```
revenues = num_sold_products * product_prices
```

```
revenues_T = revenues.T
```

```
gross_profits = revenues_T * profit_margin_of_shops
```

	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	298 400.	60 000.	156 800.	5 600.
"Raincoat"	1 599 750.	409 500.	38 250.	6 750.
"Sweater"	93 100.	393 300.	45 600.	182 400.

×

	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78	
0.45	0.17	0.52	0.78	
0.45	0.17	0.52	0.78	

➡

	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
134 280.0	10 200.0	81 536.0	4 368.0	
719 887.5	69 615.0	19 890.0	5 265.0	
41895.0	66 861.0	23 712.0	142 272.0	

$(n_{\text{product_types}}, n_{\text{shops}}) \vee (n_{\text{shops}},) \rightarrow (n_{\text{product_types}}, n_{\text{shops}})$

Exercise - Total profit

...

```
gross_profit_per_shop = np.sum(gross_profits, axis=0)
```

	"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	134 280.0	10 200.0	81 536.0	4 368.0
"Raincoat"	719 887.5	69 615.0	19 890.0	5 265.0
"Sweater"	41895.0	66 861.0	23 712.0	142 272.0



`np.sum(..., axis=0)`

`(n_product_types, n_shops) →`

Exercise - Total profit

...

```
gross_profit_per_shop = np.sum(gross_profits, axis=0)
```

	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
"Swimming suit"	134 280.0	10 200.0	81 536.0	4 368.0
"Raincoat"	719 887.5	69 615.0	19 890.0	5 265.0
"Sweater"	41895.0	66 861.0	23 712.0	142 272.0



`np.sum(..., axis=0)`

896 062.5	146 676.0	125 138.0	151 905.0
-----------	-----------	-----------	-----------

$(n_product_types, n_shops) \rightarrow (n_shops,)$

Exercise - Total profit

...

```
gross_profit_per_shop = np.sum(gross_profits, axis=0)
```

```
profit_per_shop = gross_profit_per_shop - cost_of_shops
```

"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"		"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"		"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
896 062.5	146 676.0	125 138.0	151 905.0	-	91 250.	275 000.	78 000.	45 700.	→	804 812.5	-128 324.0	47 138.0	106 205.0

$(n_shops,) \vee (n_shops,) \rightarrow (n_shops,)$

Exercise - Total profit

...

```
profit_per_shop = gross_profit_per_shop - cost_of_shops
```

```
total_profit_no_hq = np.sum(profit_per_shop)
```

"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
804 812.5	-128 324.0	47 138.0	106 205.0



829 831.46

(n_shops,) → scalar

np.sum(...)

```
total_profit = total_profit_no_hq - cost_of_hq
```

829 831.46 - 145 000.



684 831.46

scalar v scalar → scalar

An alternative approach to transpose

Exercise - Total profit (v2)

```
revenues = num_sold_products * product_prices
```

```
gross_profits = revenues * profit_margin_of_shops
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	298 400.	1 599 750.	93 100.
"Reykjavík"	60 000.	409 500.	393 300.
"Tunis"	156 800.	38 250.	45 600.
"Ulaanbaatar"	5 600.	6 750.	182 400.

???

×

"Hanoi"	"Reykjavík"	"Tunis"	"Ulaanbaatar"
0.45	0.17	0.52	0.78

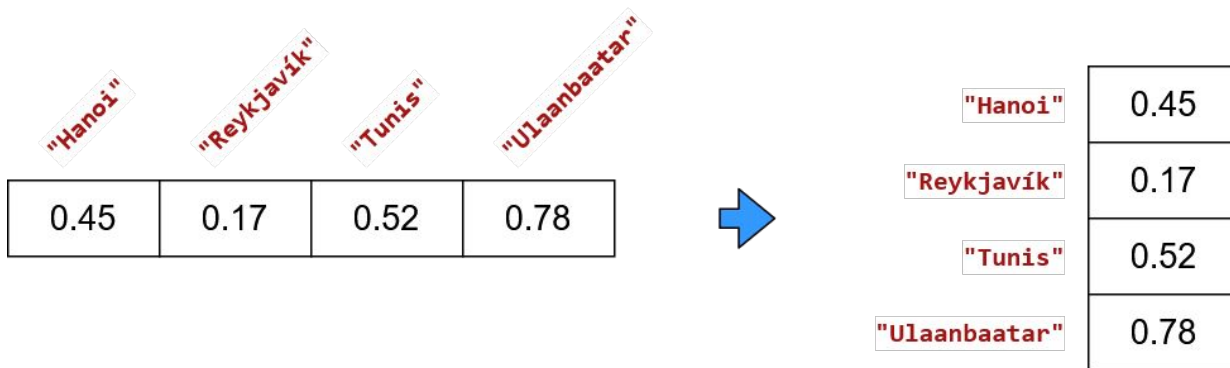
(n_shops, n_product_types) v (n_shops,) → X

Exercise - Total profit (v2)

```
revenues = num_sold_products * product_prices
```

```
profit_margin_of_shops_2d = profit_margin_of_shops.reshape(-1, 1)
```

```
#profit_margin_of_shops_2d = profit_margin_of_shops_2d[:, None]      # same
```



$(n_shops,) \rightarrow (n_shops, 1)$

NumPy - Reshaping arrays, np.newaxis / None index

```
a = np.arange(6, dtype=np.int64)                # a.shape == (6,)

print( a[np.newaxis, :].shape )                 # (1, 6)
print( a[None, :].shape )                      # (1, 6) -- same as the previous line: None is
                                                # an alias for np.newaxis in this context
print( a[None, :] )                            # [[0 1 2 3 4 5]]

print( a[ None, [2, 4, 4], None ].shape )      # (1, 4, 1)
print( a[ None, [2, 4, 4], None ] )           # [[[2]
                                                #    [4]
                                                #    [4]]]
```

Syntactic sugar: An np.newaxis / None index is equivalent to reshaping by adding a new axis with length 1 at the appropriate position.

NumPy - Reshaping arrays, np.newaxis / None index

```
a = np.arange(6, dtype=np.int64)                # a.shape == (6,)

print( a[np.newaxis, :].shape )                  # (1, 6)
print( a[None, :].shape )                        # (1, 6) -- same as the previous line: None is
                                                # an alias for np.newaxis in this context

print( a[None, :] )                             # [[0 1 2 3 4 5]]

print( a[ None, [2, 4, 4], None ].shape )        # (1, 4, 1)
print( a[ None, [2, 4, 4], None ] )              # [[[2]
                                                #      [4]
                                                #      [4]]]
```



Equivalent to: `a[ [2, 4, 4] ].reshape(1, -1, 1)`

Syntactic sugar: An `np.newaxis` / `None` index is equivalent to reshaping by adding a new axis with length 1 at the appropriate position.

Exercise - Total profit (v2)

```
revenues = num_sold_products * product_prices  
profit_margin_of_shops_2d = profit_margin_of_shops.reshape(-1, 1)  
  
gross_profits = revenues * profit_margin_of_shops_2d
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	298 400.	1 599 750.	93 100.
"Reykjavík"	60 000.	409 500.	393 300.
"Tunis"	156 800.	38 250.	45 600.
"Ulaanbaatar"	5 600.	6 750.	182 400.

×

0.45
0.17
0.52
0.78

(n_shops, n_product_types) v (n_shops, 1)

An alternative approach to transpose

Exercise - Total profit (v2)

```
revenues = num_sold_products * product_prices  
profit_margin_of_shops_2d = profit_margin_of_shops.reshape(-1, 1)  
  
gross_profits = revenues * profit_margin_of_shops_2d
```

	Swimming suit	Raincoat	Sweater
Hanoi	298 400.	1 599 750.	93 100.
Reykjavík	60 000.	409 500.	393 300.
Tunis	156 800.	38 250.	45 600.
Ulaanbaatar	5 600.	6 750.	182 400.

×

Swimming suit	Raincoat	Sweater
0.45	0.45	0.45
0.17	0.17	0.17
0.52	0.52	0.52
0.78	0.78	0.78

→

Swimming suit	Raincoat	Sweater
134 280.0	719 887.5	41895.0
10 200.0	69 615.0	66 861.0
81 536.0	19 890.0	23 712.0
4 368.0	5 265.0	142 272.0

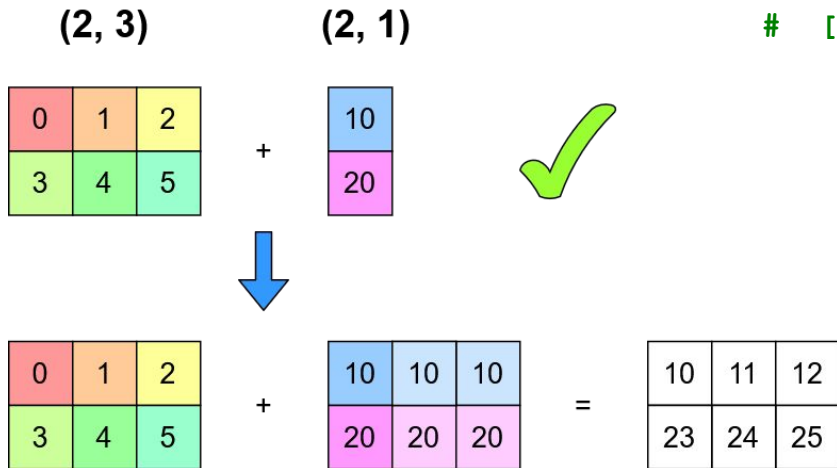
$(n_shops, n_product_types) \times (n_shops, 1) \rightarrow (n_shops, n_product_types)$

NumPy - Element-wise ops & broadcasting, (n, m) vs. (n, 1)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20], dtype=np.int64)                 # b.shape == (2,)

b2d = b[:, None]                                       # same as b2d = b.reshape(-1, 1)    # b2d.shape == (2, 1)
print(a + b2d)                                         # [[10 11 12]
                                                         #  [23 24 25]]
```



NumPy - Rules of broadcasting

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1 to the shape with the shorter length.
2. Check the **lengths of axes** at matching positions!
 - a. If they are equal → **OK (no action)**
 - b. If they are not equal, but one of them is 1
→ **Perform virtual repetition along this axis to match the axis lengths.**
 - c. If they are not equal, and none of them is 1
→ **Broadcasting fails.**

Example:
(2, 3) v (2, 1)

(2, 3) v (2, 1)

(2, 3) v (2, 1)

→ (2, 3) v (2, 3)

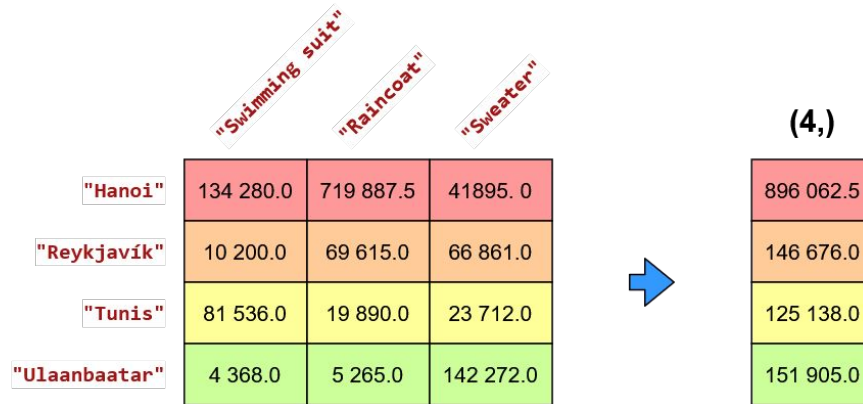


Broadcasting succeeds!
Common shape: (2, 3)

Exercise - Total profit (v2)

...

```
gross_profit_per_shop = np.sum(gross_profits, axis=1)
```



	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	134 280.0	719 887.5	41895. 0
"Reykjavík"	10 200.0	69 615.0	66 861.0
"Tunis"	81 536.0	19 890.0	23 712.0
"Ulaanbaatar"	4 368.0	5 265.0	142 272.0

→

(4,)
896 062.5
146 676.0
125 138.0
151 905.0

`np.sum(..., axis=1)`

$(n_shops, n_product_types) \rightarrow (n_shops,)$

Exercise - Total profit (v2)

...

```
gross_profit_per_shop = np.sum(gross_profits, axis=1)
```

```
profit_per_shop = gross_profit_per_shop - cost_of_shops
```

"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"		"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"		"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
896 062.5	146 676.0	125 138.0	151 905.0	-	91 250.	275 000.	78 000.	45 700.	→	804 812.5	-128 324.0	47 138.0	106 205.0

(n_shops,) v (n_shops,) → (n_shops,)

Exercise - Total profit (v2)

...

```
profit_per_shop = gross_profit_per_shop - cost_of_shops  
total_profit_no_hq = np.sum(profit_per_shop)
```

"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
804 812.5	-128 324.0	47 138.0	106 205.0



829 831.46

(n_shops,) → scalar

np.sum(...)

```
total_profit = total_profit_no_hq - cost_of_hq
```

829 831.46 - 145 000.



684 831.46

scalar v scalar → scalar

Exercise - Number of products to sell for break-even

`gross_profits_per_product_shop = product_prices * profit_margin_of_shops`

Need to calculate: How much (gross) profit do we earn after selling each product in each shop?

"Swimming suit"	"Raincoat"	"Sweater"	???	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
800.	2 250.	1 900.	x	0.45	0.17	0.52	0.78

$(n_shops,) \vee (n_product_types,) \rightarrow \mathbf{X}$

Exercise - Number of products to sell for break-even

`gross_profits_per_product_shop = product_prices * profit_margin_of_shops`

Need to calculate: How much (gross) profit do we earn after selling each product in each shop?

Need to compare all possible pairs of elements from the two arrays!

"Swimming suit"	"Raincoat"	"Sweater"	???	"Hanoi"	"Reykjavik"	"Tunis"	"Ulaanbaatar"
800.	2 250.	1 900.	x	0.45	0.17	0.52	0.78

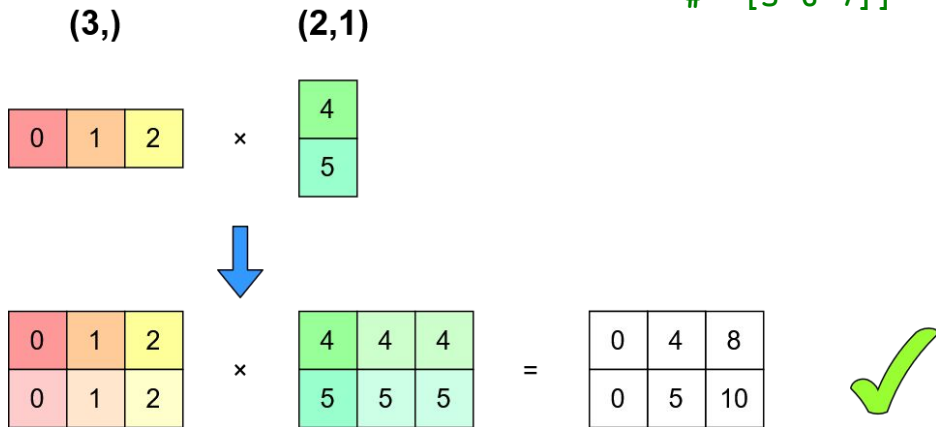
$(n_{\text{product_types}}) \vee (n_{\text{shops}}) \rightarrow \mathbf{X}$

NumPy - Element-wise ops & broadcasting, (1, m) vs. (n, 1)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.arange(3, dtype=np.int64)           # a.shape == (3,)
b = np.arange(2, dtype=np.int64)+4         # b.shape == (2,)

print( (a + b[:,None]).shape )              # (2, 3)
print( a + b[:,None] )                     # [[4 5 6]
                                           #  [5 6 7]]
```



NumPy - Element-wise ops & broadcasting, (1, m) vs. (n, 1)

Broadcasting is the **virtual repetition of elements** to match the shapes of two or more arrays.

```
a = np.arange(3, dtype=np.int64) # a.shape == (3,)
```

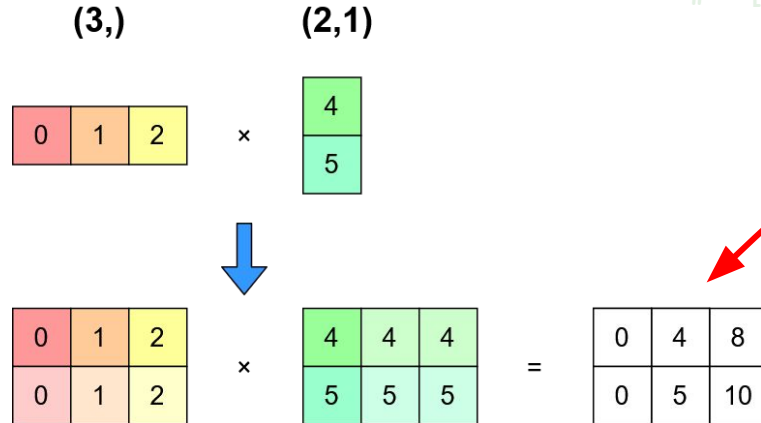
```
b = np.arange(2, dtype=np.int64)+4 # b.shape == (2,)
```

Useful for: Applying an operation on all possible combinations of the elements from two arrays.

```
print( (a + b[:,None]).shape) # (2, 3)
```

```
print( a + b[:,None] ) # [[4 5 6]
```

```
# [5 6 7]]
```



E.g.: $0 \times 4 = 0$
 $1 \times 4 = 4$
...
 $2 \times 5 = 10$



NumPy - Rules of broadcasting

Example:
(3,) v (2,1)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(1,3) v (2,1)

2. Check the **lengths of axes** at matching positions!

- a. If they are equal → **OK (no action)**

- b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis
to match the axis lengths.**

→ (1,3) v (2,3)

→ (2,3) v (2,3)

- c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**



Broadcasting succeeds!
Common shape: (2, 3)

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]
```

"Swimming suit"	"Raincoat"	"Sweater"		
800.	2 250.	1 900.	×	
				0.45
				0.17
				0.52
				0.78
				"Hanoi"
				"Reykjavík"
				"Tunis"
				"Ulaanbaatar"

(n_product_types,) v (n_shops, 1) →

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]
```

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	800.	2 250.	1 900.
"Reykjavík"	800.	2 250.	1 900.
"Tunis"	800.	2 250.	1 900.
"Ulaanbaatar"	800.	2 250.	1 900.

×

0.45	0.45	0.45
0.17	0.17	0.17
0.52	0.52	0.52
0.78	0.78	0.78

→

	"Swimming suit"	"Raincoat"	"Sweater"
360.	1 012.5	855.	
136.	382.5	323.	
416.	1 170.	988.	
624.	1 755.	1 482.	

$(n_product_types,) \vee (n_shops, 1) \rightarrow (n_shops, n_product_types)$

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]  
num_products_to_cover_shop_cost = cost_of_shops[:, None] / gross_profits_per_product_shop
```

"Hanoi"	91 250.
"Reykjavík"	275 000.
"Tunis"	78 000.
"Ulaanbaatar"	45 700.

/

	"Swimming suit"	"Raincoat"	"Sweater"
	360.	1 012.5	855.
	136.	382.5	323.
	416.	1 170.	988.
	624.	1 755.	1 482.

(n_shops, 1) v (n_shops, n_product_types) →

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]
num_products_to_cover_shop_cost = cost_of_shops[:, None] / gross_profits_per_product_shop
```

The diagram illustrates the calculation of the average of four cities' values across three categories. It is divided into three main sections by a large blue arrow pointing from left to right.

Left Section (Input Data):

	"Swimming suit"	"Raincoat"	"Sweater"
"Hanoi"	91 250.	91 250.	91 250.
"Reykjavík"	275 000.	275 000.	275 000.
"Tunis"	78 000.	78 000.	78 000.
"Ulaanbaatar"	45 700.	45 700.	45 700.

Middle Section (Separator):

A large blue diagonal line (/) separates the input data from the output data.

Right Section (Output Data):

	"Swimming suit"	"Raincoat"	"Sweater"
Average	253.47	90.12	106.72
Standard deviation	2 022.1	718.95	851.39
Minimum	187.5	66.67	78.95
Maximum	73.24	26.04	30.84

$(n_shops, 1) \vee (n_shops, n_product_types) \rightarrow (n_shops, n_product_types)$

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]  
num_products_to_cover_shop_cost = cost_of_shops[:, None] / gross_profits_per_product_shop  
num_products_to_cover_shop_cost_i = np.ceil(num_products_to_cover_shop_cost).astype(np.int64)
```

"Swimming suit"	"Raincoat"	"Sweater"
253.47	90.12	106.72
2 022.1	718.95	851.39
187.5	66.67	78.95
73.24	26.04	30.84



"Swimming suit"	"Raincoat"	"Sweater"
254	91	107
2 023	719	852
188	67	79
74	27	31

$(n_shops, n_product_types) \rightarrow (n_shops, n_product_types)$

Exercise - Number of products to sell for break-even

```
gross_profits_per_product_shop = product_prices * profit_margin_of_shops[:, None]  
num_products_to_cover_shop_cost = cost_of_shops[:, None] / gross_profits_per_product_shop  
num_products_to_cover_shop_cost_i = np.ceil(num_products_to_cover_shop_cost).astype(np.int64)
```



"Swimming suit"	"Raincoat"	"Sweater"
253.47	90.12	106.72
2 022.1	718.95	851.39
187.5	66.67	78.95
73.24	26.04	30.84

"Swimming suit"	"Raincoat"	"Sweater"
254	91	107
2 023	719	852
188	67	79
74	27	31

**Datatype (dtype)
conversion**

$(n_shops, n_product_types) \rightarrow (n_shops, n_product_types)$

NumPy - Datatype conversion

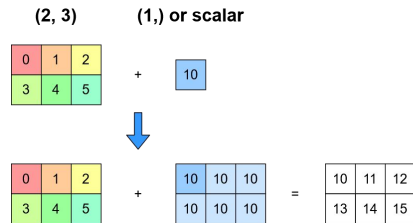
```
a = np.linspace(0., 5., num=7, endpoint=True, dtype=np.float32)
print(a.dtype)           # float32
print(a)                 # [0.          0.83333333 1.6666666 2.5          3.3333333 4.1666665 5.          ]

a_i = a.astype(np.int64)
print(a_i.dtype)         # int64
print(a_i)               # [0 0 1 2 3 4 5]

a_b = a.astype(np.bool_)
print(a_b.dtype)         # bool
print(a_b)               # [False  True  True  True  True  True  True]
```

`ndarray.astype()`: If dtype conversion happens, a new array is returned.

NumPy - Rules of broadcasting



Example:
(3, 2) v (1,)

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

(3, 2) v (1, 1)

2. Check the **lengths of axes** at matching positions!

a. If they are equal → **OK (no action)**

b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis**
to match the axis lengths.

c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**

→ **(3, 2) v (1, 2)**

→ **(3, 2) v (3, 2)**



Broadcasting succeeds!
Common shape: (3, 2)

NumPy - Rules of broadcasting

(2, 3) (1,) or scalar

0	1	2
3	4	5

+

10



0	1	2
3	4	5

+

10	10	10
10	10	10

=

10	11	12
13	14	15

Same for scalars
(at least logically...)

Example:
 $(3, 2) \vee (1,)$

The algorithm of shape matching

1. Check the **number of dimensions** of the arguments!
Until they do not match, add leading axes of length 1
to the shape with the shorter length.

$(3, 2) \vee (1, 1)$

2. Check the **lengths of axes** at matching positions!

a. If they are equal → **OK (no action)**

b. If they are not equal, but one of them is 1

→ **Perform virtual repetition along this axis**
to match the axis lengths.

c. If they are not equal, and none of them is 1

→ **Broadcasting fails.**

→ $(3, 2) \vee (1, 2)$

→ $(3, 2) \vee (3, 2)$



Broadcasting succeeds!
Common shape: (3, 2)

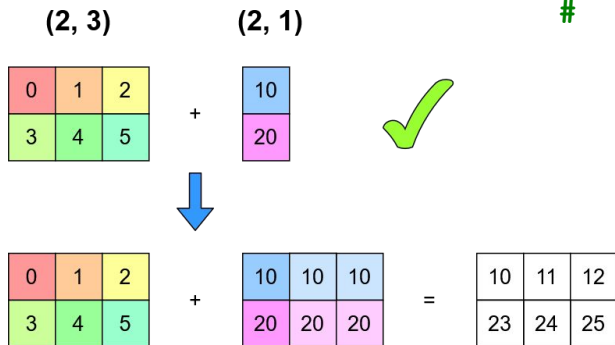
NumPy - Broadcasting happens when...

Automatic broadcasting happens...

- Before element-wise operations to match argument shapes.

```
a = np.array([[0, 1, 2], [3, 4, 5]], dtype=np.int64)    # a.shape == (2, 3)
b = np.array([10, 20], dtype=np.int64)                  # b.shape == (2,)
```

```
print( a + b[:, None] )                                # [[10 11 12]
                                                         #  [23 24 25]]
```



NumPy - Broadcasting happens when...

Automatic broadcasting happens...

- Before element-wise operations to match argument shapes.
- Before an array-writing assignment to match the shapes on the two sides.

```
a = np.arange(12).reshape(3, 4) * 10      # a.shape == (3, 4)
```

```
a[ ::2, [0,3,1] ] = [[42], [99]]
```

```
print(a)
```

```
# [[ 42  42  20  42]
#   [ 40  50  60  70]
#   [ 99  99 100  99]]
```

(2, 3) v (2, 1)



NumPy - Broadcasting happens when...

Automatic broadcasting happens...

- Before element-wise operations to match argument shapes.
- Before an array-writing assignment to match the shapes on the two sides.
- In certain cases of indexing.

```
a = np.arange(6).reshape(2, 3) * 10      # a.shape == (2, 3)
print(a)                                # [[ 0 10 20]
                                         #  [30 40 50]]
```

```
b = a[ np.array([1,0,1])[:, None], [2,1,2,0] ]
print(b.shape)                           # (3, 4)
print(b)                                # [[50 40 50 30]
                                         #  [20 10 20  0]
                                         #  [50 40 50 30]]
```

(3, 1) v (4,)



NumPy - Broadcasting happens when...

Automatic broadcasting happens...

- Before element-wise operations to match argument shapes.
- Before an array-writing assignment to match the shapes on the two sides.
- In certain cases of indexing.
- Various other NumPy functions support broadcasting.

```
v = np.array([42, 99], dtype=np.int64)
a = np.full((2, 3), dtype=np.float32, fill_value=v[:, None])
```

(2, 3) v (2, 1)



```
print(a)
```

```
# [[42. 42. 42.]
```

```
# [99. 99. 99.]]
```

NumPy - Finding index of **all** elements with a condition (nD)

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64)           # a.shape == (2, 3)
my_mask = a % 20 == 0

print(my_mask)              # [[ True False  True]
                             #  [False  True False]]

print(my_mask.shape)        # (2, 3)

print(my_mask.dtype)        # bool
```

```
my_idxs_tuple = np.where(my_mask)
print(my_idxs_tuple)         # (array([0, 0, 1]), array([0, 2, 1]))
print(type(my_idxs_tuple))  # <class 'tuple'>

my_idxs_arr0, my_idxs_arr1 = my_idxs_tuple
print(my_idxs_arr0)          # [0 0 1]
print(my_idxs_arr1)          # [0 2 1]
```

np.where: Converting a mask to integer indices

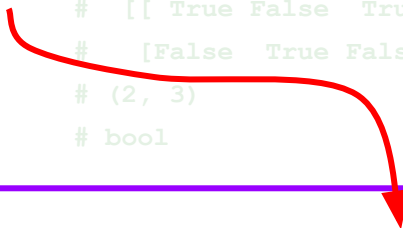
A use case: Find the index of all elements with a condition / mask!

NumPy - Finding index of **all** elements with a condition (nD)

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64) # a.shape == (2, 3)
```

A tuple with integer index arrays where the indices are **separated by axis** are the standard format of multi-dimensional indices.

```
print(my_mask) # [[ True False  True]
               #  [False  True False]]
print(my_mask.shape) # (2, 3)
print(my_mask.dtype) # bool
```



```
my_idx_tuple = np.where(my_mask)
print(my_idx_tuple) # (array([0, 0, 1]), array([0, 2, 1]))
print(type(my_idx_tuple)) # <class 'tuple'>

my_idx_arr0, my_idx_arr1 = my_idx_tuple
print(my_idx_arr0) # [0 0 1]
print(my_idx_arr1) # [0 2 1]
```

np.where: Converting a mask to integer indices

A use case: Find the index of all elements with a condition / mask!

NumPy - Finding index of **all** elements with a condition (nD)

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64) # a.shape == (2, 3)
```

A tuple with integer index arrays where the indices are **separated by axis** are the standard format of multi-dimensional indices.

```
print(my_mask) # [[ True False  True]
```

```
# [False  True False]]
```

The tuple can be directly used in an indexing (Indexing case [**3** , **3**])

```
print(my_mask.dtype) # bool
```

```
my_idxs_tuple = np.where(my_mask)
print(my_idxs_tuple) # (array([0, 0, 1]), array([0, 2, 1]))
print(type(my_idxs_tuple)) # <class 'tuple'>
```

```
my_idxs_arr0, my_idxs_arr1 = my_idxs_tuple
print(my_idxs_arr0) # [0 0 1]
print(my_idxs_arr1) # [0 2 1]
```

```
print( a[my_idxs_tuple] ) # [ 0 20 40]
```

NumPy - Finding index of **all** elements with a condition (nD)

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64)           # a.shape == (2, 3)
my_mask = a % 20 == 0

print(my_mask)               # [[ True False  True]
                             #  [False  True False]]

print(my_mask.shape)         # (2, 3)

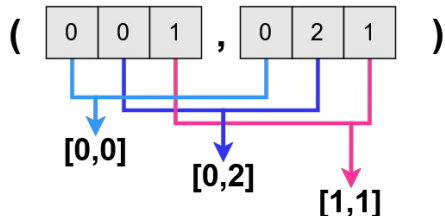
print(my_mask.dtype)         # bool
```

my_mask.shape: (2, 3)

True	False	True
False	True	False

np.where(my_mask)

np.nonzero(my_mask)



np.argwhere(my_mask)

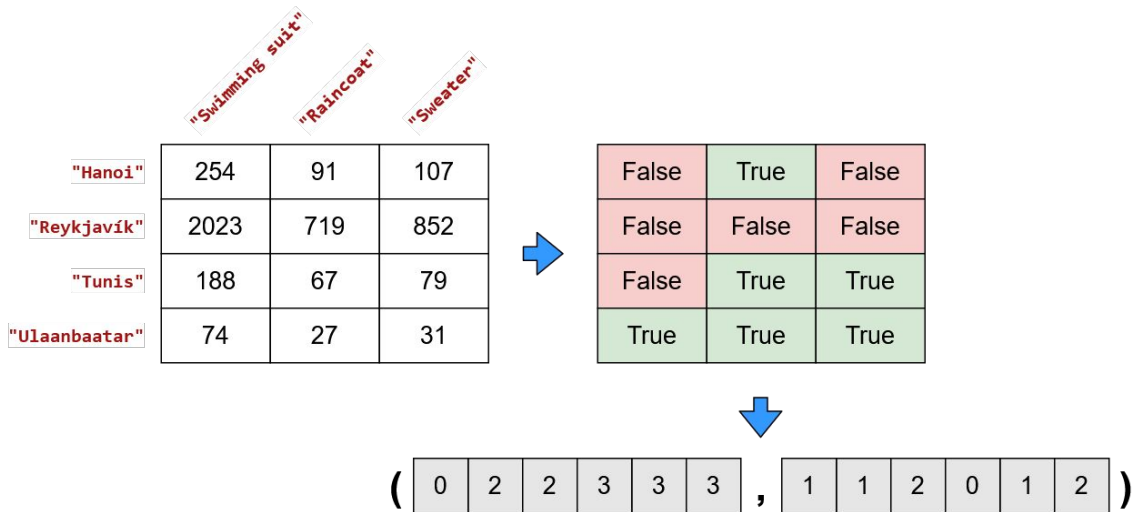
0	0	→ [0,0]
0	2	→ [0,2]
1	1	→ [1,1]

NumPy - Finding index of **all** elements with a condition (nD)

```
mask_where_condition = num_products_to_cover_shop_cost_i < 100      # (n_shops, n_product_types) bool_
```

```
idxs_tup_where_condition = np.where(mask_where_condition)           # tuple of 1D arrays
```

```
idxs0_where_condition, idxs1_where_condition = idxs_tup_where_condition  # unpack tuple into the two arrays
```



NumPy - Finding index of the **first** element with a condition

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64)           # a.shape == (2, 3)
my_mask = a > 35

print(my_mask)              # [[False False False]
                             #  [False  True  True]]

print(my_mask.shape)        # (2, 3)

print(my_mask.dtype)        # bool
```

```
first_flat_idx = np.argmax(my_mask)
print(first_flat_idx)        # 4
# print(my_mask[first_flat_idx]) # Incorrect: my_mask[4,:]
print(my_mask.reshape(-1)[first_flat_idx]) # True

print(a.reshape(-1)[first_flat_idx])      # 40
```

np.argmax: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64) # a.shape == (2, 3)
```

`np.argmax()` returns a flat index when used on a multi-dimensional array without an axis parameter.

```
print(my_mask) # [[False False False]
```

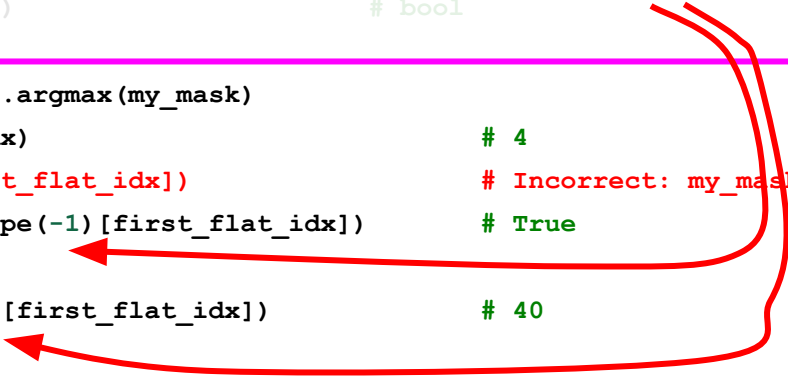
```
# [False True True]]
```

A flat index is indexing the 1D reshaped view of the array.

```
print(my_mask.dtype) # bool
```

```
first_flat_idx = np.argmax(my_mask)
print(first_flat_idx) # 4
# print(my_mask[first_flat_idx]) # Incorrect: my_mask[4,:]
print(my_mask.reshape(-1)[first_flat_idx]) # True

print(a.reshape(-1)[first_flat_idx]) # 40
```



`np.argmax`: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Finding index of the **first** element with a condition

```
a = np.array([[0, 10, 20], [30, 40, 50]], dtype=np.int64)           # a.shape == (2, 3)
my_mask = a > 35

print(my_mask)                                                       # [[False False False]
                                                                    # [False True True]]

print(my_mask.shape)                                                 # (2, 3)

print(my_mask.dtype)                                                 # bool
```

```
first_flat_idx = np.argmax(my_mask)
print(first_flat_idx)                                                # 4
first_idx_tuple = np.unravel_index(first_flat_idx, shape=my_mask.shape)
print(first_idx_tuple)                                               # (1, 1)
print(my_mask[first_idx_tuple])                                     # True

print(a[first_idx_tuple])                                           # 40
```

An alternative: Converting flat indices to multi-dimensional indices.

np.argmax: Finding the index of the largest element

A use case: Find the index of the first element with a condition / mask!

NumPy - Flat indices $\longleftrightarrow$ Multi-dimensional indices

my_mask.shape: (4, 3)

False	True	False
False	False	False
False	True	True
True	True	True

